

Data Analytics Project Report

SQL DATA EXTRACTION, AGGREGATION, AND IN-MEMORY SIMULATION PIPELINE

Academic Board: DecodeLabs Internship Panel

Document Type: Technical Project Report

Status: **Verified & Approved**

1. Methodology and Analytical Tools

This technical report documents the engineering design and computational implementation of an automated backend data pipeline. The system architecture securely ingests commercial raw records from an Excel spreadsheet and converts them into a flexible, volatile database environment through the integration of three industry-standard technologies:

- **Python Core Environment:** Serves as the central high-level infrastructure layer. It automates localized file path detection by overriding native interface layers using Tkinter dialog controls.
- **Pandas Aggregation Framework:** Acts as the data parsing and manipulation engine. It dynamically maps matrix records, attributes, and indexes into highly efficient in-memory variables known as DataFrames.
- **SQLite3 (In-Memory Database Configuration):** A zero-latency, serverless database engine initialized explicitly inside volatile RAM storage (:memory:). This configuration facilitates fast data transformation from DataFrames to a physical query table (SalesTable), permitting complex structural SQL logic optimization without infrastructure dependencies.

2. Complete Pipeline Source Code

The complete functional Python script implemented to execute the pipeline operations, manage UI manual file selections, and print structured analytical outputs is provided below:

```

import sqlite3
import pandas as pd
import tkinter as tk
from tkinter import filedialog

root = tk.Tk()
root.withdraw()
root.attributes('-topmost', True)
file_path = filedialog.askopenfilename(
    title="Select Your Excel File",
    filetypes=[("Excel files", "*.xlsx *.xls")]
)

df = pd.read_excel(file_path)

conn = sqlite3.connect(':memory:')

df.to_sql('SalesTable', conn, index=False, if_exists='replace')

def run_query(query_title, query):
    print(f"\n=====")
    print(f" {query_title}")
    print(f"=====")
    result = pd.read_sql_query(query, conn)
    print(result.to_string(index=False))

run_query("Data Extraction (SELECT)", """
SELECT OrderID, CustomerID, Product, Quantity, TotalPrice
FROM SalesTable
LIMIT 10;
""")

run_query("WHERE Clause - Equality (Desk Only)", """
SELECT OrderID, Product, Quantity, TotalPrice
FROM SalesTable
WHERE Product = 'Desk'
LIMIT 10;
""")

run_query("WHERE Clause - Comparison (TotalPrice > 1500)", """
SELECT OrderID, Product, Quantity, TotalPrice
FROM SalesTable
WHERE TotalPrice > 1500.00
LIMIT 10;
""")

```

```

run_query("Pattern Matching (ReferralSource LIKE 'In%')", "")
SELECT OrderID, Product, ReferralSource, TotalPrice
FROM SalesTable
WHERE ReferralSource LIKE 'In%'
LIMIT 10;
""")

run_query("GROUP BY & SUM (Total Revenue per Product)", "")
SELECT Product, SUM(TotalPrice) AS TotalRevenue
FROM SalesTable
GROUP BY Product;
""")

run_query("COUNT & AVG Aggregations per Payment Method", "")
SELECT PaymentMethod, COUNT(OrderID) AS TotalOrders, AVG(TotalPrice) AS
AverageOrderValue
FROM SalesTable
GROUP BY PaymentMethod;
""")

run_query("ORDER BY (Products Ranked by Revenue DESC)", "")
SELECT Product, SUM(Quantity) AS TotalUnitsSold, SUM(TotalPrice) AS TotalRevenue
FROM SalesTable
GROUP BY Product
ORDER BY TotalRevenue DESC;
""")

conn.close()

```

3. SQL Operations, Logical Rationales & Real Terminal Output Streams

Step 1: Exploratory Data Extraction

Technical Rationale: Performs a baseline horizontal query to retrieve a 10-row slice from the mapped table schema. This acts as a structural validation to verify that variable fields, column definitions, and indices successfully migrated from Excel into SQLite tables.

SQL Query Injected:

```

SELECT OrderID, CustomerID, Product, Quantity, TotalPrice
FROM SalesTable
LIMIT 10;

```

Real Console Terminal Output:

```
=====
📊 Data Extraction (SELECT)
=====

  OrderID CustomerID Product  Quantity  TotalPrice
ORD200000      C72649 Monitor      5      2853.10
ORD200001      C75739  Phone      2       302.70
ORD200002      C81728 Tablet      5      2753.40
ORD200003      C33540  Chair      1       273.19
ORD200004      C81840 Printer     4      2504.04
ORD200005      C37249  Phone      2       491.72
ORD200006      C83492 Laptop      1       664.42
ORD200007      C41460 Monitor     5       747.75
ORD200008      C26817  Phone      2       268.56
ORD200009      C31946   Desk      4      2037.52
```

Step 2: Nominal Conditional Filtering

Technical Rationale: Implements textual strict-matching parameters inside a `WHERE` clause to filter the dataset horizontally. This query narrows the analysis down exclusively to transactions tracking the item token 'Desk'.

SQL Query Injected:

```
SELECT OrderID, Product, Quantity, TotalPrice
FROM SalesTable
WHERE Product = 'Desk'
LIMIT 10;
```

Real Console Terminal Output:

```
=====
📊 WHERE Clause - Equality (Desk Only)
=====

  OrderID Product  Quantity  TotalPrice
ORD200009   Desk      4      2037.52
ORD200018   Desk      1       431.44
ORD200029   Desk      3     1270.59
ORD200040   Desk      2       688.04
ORD200047   Desk      2       419.88
ORD200060   Desk      1       417.10
ORD200061   Desk      4       488.80
ORD200062   Desk      2        86.30
ORD200070   Desk      1       629.21
ORD200088   Desk      3       344.43
```

Step 3: Comparative Financial Threshold Filtering

Technical Rationale: Applies comparative logical parameters (>) to target specific invoice limits. This isolates luxury consumer behavior profiles where transaction total prices cross the \$1500.00 floor margin.

SQL Query Injected:

```
SELECT OrderID, Product, Quantity, TotalPrice
FROM SalesTable
WHERE TotalPrice > 1500.00
LIMIT 10;
```

Real Console Terminal Output:

```
=====
 WHERE Clause - Comparison (TotalPrice > 1500)
=====
  OrderID Product  Quantity  TotalPrice
ORD200000 Monitor      5    2853.10
ORD200002 Tablet      5    2753.40
ORD200004 Printer     4    2504.04
ORD200009 Desk        4    2037.52
ORD200010 Tablet      5    3129.85
ORD200031 Chair       3    1500.24
ORD200032 Tablet      5    2683.60
ORD200034 Chair       4    2306.08
ORD200041 Monitor     3    1834.35
ORD200046 Monitor     4    2200.56
```

Step 4: Substring Pattern Matching Auditing

Technical Rationale: Invokes pattern matching features using the `LIKE` command linked with the `%` character token. This enables aggregated analysis of any user referral channel starting with 'In' (e.g., Instagram).

SQL Query Injected:

```
SELECT OrderID, Product, ReferralSource, TotalPrice
FROM SalesTable
WHERE ReferralSource LIKE 'In%'
LIMIT 10;
```

Real Console Terminal Output:

```
=====
📊 Pattern Matching (ReferralSource LIKE 'In%')
=====

  OrderID Product ReferralSource TotalPrice
ORD200000 Monitor      Instagram    2853.10
ORD200005  Phone      Instagram     491.72
ORD200010 Tablet      Instagram    3129.85
ORD200014 Tablet      Instagram     786.66
ORD200020 Chair       Instagram     312.55
ORD200037 Laptop      Instagram     153.48
ORD200040 Desk        Instagram     688.04
ORD200041 Monitor      Instagram    1834.35
ORD200042 Printer      Instagram     303.32
ORD200043 Chair       Instagram    1497.63
```

Step 5: Aggregation and Financial Consolidation

Technical Rationale: Segments independent invoice rows by mapping distinct catalog keys (`GROUP BY Product`) and computing an arithmetic accumulation macro (`SUM`) over total order prices to determine absolute calculated revenue results per product line.

SQL Query Injected:

```
SELECT Product, SUM(TotalPrice) AS TotalRevenue
FROM SalesTable
GROUP BY Product;
```

Real Console Terminal Output:

```
=====
📊 GROUP BY & SUM (Total Revenue per Product)
=====

Product TotalRevenue
Chair    195620.11
Desk     167459.93
Laptop   192126.56
Monitor  175651.41
Phone    151722.39
Printer  195612.61
Tablet   186568.95
```

Step 6: Multidimensional Operational Audits

Technical Rationale: Collects user check-out fields grouped by specific transaction parameters (`PaymentMethod`). It processes a count tally (`COUNT`) alongside average value equations (`AVG`) to isolate client processing platform trends.

SQL Query Injected:

```
SELECT PaymentMethod, COUNT(OrderID) AS TotalOrders, AVG(TotalPrice) AS  
AverageOrderValue  
FROM SalesTable  
GROUP BY PaymentMethod;
```

Real Console Terminal Output:

```
=====
📊 COUNT & AVG Aggregations per Payment Method
=====
PaymentMethod  TotalOrders  AverageOrderValue
      Cash           246         1056.041829
Credit Card     234         1127.553974
Debit Card      232         1001.556810
Gift Card       230         1070.973565
      Online       258         1017.220698
```

Step 7: Hierarchical Performance Ranking Evaluation

Technical Rationale: Aggregates operational catalog metrics, counts units sold, calculates absolute financial revenue values, and references dynamic inverse ranking algorithms (`ORDER BY DESC`) to sort items from highest revenue to lowest.

SQL Query Injected:

```
SELECT Product, SUM(Quantity) AS TotalUnitsSold, SUM(TotalPrice) AS TotalRevenue  
FROM SalesTable  
GROUP BY Product  
ORDER BY TotalRevenue DESC;
```

Terminal Output:

```
=====
📊 ORDER BY (Products Ranked by Revenue DESC)
=====
Product  TotalUnitsSold  TotalRevenue
Chair      562      195620.11
Printer    542      195612.61
Laptop     535      192126.56
Tablet     497      186568.95
Monitor    480      175651.41
Desk       508      167459.93
Phone      411      151722.39
```

4. Key Analytical Insights

- **Core Revenue Vector:** Relational matrix outputs show that **Chairs** achieved peak commercial performance, leading in volume with **562 total units sold** and generating maximum revenue of **\$195,620.11**, followed closely by **Printers** (\$195,612.61).
- **Transactional Method Analysis:** Operational metrics confirm that transactions cleared via **Credit Cards** hold the highest ticket size average, registering at **\$1,127.55** per transaction, while **Online** checkout serves as the densest volume channel with **258 individual orders**.